The University of Texas at Austin

UTensil

Dylan Smith, Alex Chen, Botao Yuan

NAC 2026

March 22, 2026

Contest (1)

template.cpp

14 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define pb push_back
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
  cin.tie(0)->sync_with_stdio(0);
}
```

.bashrc

10 lines

```bash
run () {
  ok=1
  if [[ ! -f $1 || $1 -ot $1.cpp ]]
  then
    g++ $1.cpp -O2 -o $1 -std=c++17 -Wall -Wextra -Wshadow
        -Wconversion -fsanitize=undefined,address || ok=0
  fi
  [[ $ok -eq 1 ]] && ./$1
}

xmodmap -e 'clear Lock' -e 'keycode 0x42 = Escape'
```

Mathematics (2)

2.1 Equations

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

$$x_i = \frac{\det A_i'}{\det A}$$

2.2 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$
$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.3 Geometry

2.3.1 Triangles

Circumradius: $R = abc/4A$

Inradius: $r = A/p$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b + c}\right)^2\right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

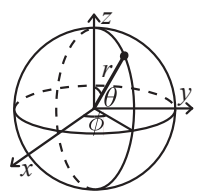
2.3.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.3.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \operatorname{acos}(z/\sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

2.4 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1 - x^2}} \qquad \frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1 - x^2}}$$
$$\frac{d}{dx} \tan x = 1 + \tan^2 x \qquad \frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) \qquad \int x e^{ax} dx = \frac{e^{ax}}{a^2}(ax - 1)$$

Integration by parts:

$$\int_a^b f(x) g(x) dx = [F(x) g(x)]_a^b - \int_a^b F(x) g'(x) dx$$

2.5 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n + 1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n + 1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$$

2.6 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$
$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$
$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$
$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.7 Probability theory

$$\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2$$
$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$
$$\text{ind. } X, Y, V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

2.7.1 Discrete distributions

Binomial distribution

$$p(k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\mathrm{Bin}(n, p)$ is approximately $\mathrm{Po}(np)$ for small p.

Geometric distribution

$$p(k) = p(1 - p)^{k-1}, k = 1, 2, \ldots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1 - p}{p^2}$$

Poisson distribution

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.7.2 Continuous distributions

Uniform distribution

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.8 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \ldots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi\mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i.

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change `null_type`.

Time: $\mathcal{O}(\log N)$

d41d8c, 16 lines

```
#include <bits/extc++.h>//d41
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;//d41

void example() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));//d41
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}//d41
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d41d8c, 7 lines

```
#include <bits/extc++.h>//d41
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C)
      ; }
};//d41
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{
    1<<16});
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").

Time: $\mathcal{O}(\log N)$

d41d8c, 30 lines

```
struct Line {//d41
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};
//d41
struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }//d41
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;//d41
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y
        ));//d41
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());//d41
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

Treap.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Time: $\mathcal{O}(\log N)$

d41d8c, 55 lines

```
struct Node {//d41
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  Node(int v) : val(v), y(rand()) {}
  void recalc();
};//d41

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

template<class F> void each(Node* n, F f) {//d41
  if (n) { each(n->l, f); f(n->val); each(n->r, f); }
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};//d41
  if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
    auto pa = split(n->l, k);
    n->l = pa.second;
    n->recalc();
    return {pa.first, n};//d41
  } else {
    auto pa = split(n->r, k - cnt(n->l) - 1); // and just "
        k"
    n->r = pa.first;
    n->recalc();
    return {n, pa.second};//d41
  }
}

Node* merge(Node* l, Node* r) {
  if (!l) return r;//d41
  if (!r) return l;
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    l->recalc();
    return l;//d41
  } else {
    r->l = merge(l, r->l);
    r->recalc();
    return r;
```

```
  }//d41
}

Node* ins(Node* t, Node* n, int pos) {
  auto pa = split(t, pos);
  return merge(merge(pa.first, n), pa.second);//d41
}

// Example application: move the range [l, r) to index k
void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c;//d41
  tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
  if (k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r));
}
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are $\mathcal{O}(\log N)$.

d41d8c, 22 lines

```
struct FT {//d41
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }//d41
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  }//d41
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >=
      sum
    // Returns n if no sum is >= sum, or -1 if empty sum is
      .
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {//d41
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }//d41
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V|\log|V| + Q)$

d41d8c, 16 lines

```
template<class T>//d41
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k)
      {
      jmp.emplace_back(sz(V) - pw * 2 + 1);//d41
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int a, int b) {//d41
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};//d41
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.
Time: $\mathcal{O}(\log N)$

d41d8c, 19 lines

```
struct Tree {//d41
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative
      fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}//d41
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)//d41
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }//d41
    return f(ra, rb);
  }
};
```

LazySegmentTree2.h

Description: Range add, set, multiply, sum
Time: $\mathcal{O}(\log N)$ per operation.

d41d8c, 56 lines

```
struct SegTree {//d41
    struct Node {
        ll sum = 0;
    };
    struct Upd {
        ll a = 1, b = 0;//d41
    };
    void apply(Node &v, Upd &u, int l, int r) {
        v.sum = v.sum * u.a + u.b * (r - l + 1);
    }
    void combine(Upd &a, Upd &b) {//d41
        a.b = a.b * b.a + b.b;
        a.a = a.a * b.a;
    }
    Node merge(Node &a, Node &b) {
        return {a.sum + b.sum};//d41
    }
    vector<Node> tree; vector<Upd> upd;
    void push(int i, int l, int r) {
        apply(tree[i], upd[i], l, r);
        if (i * 2 < sz(tree)) {//d41
            combine(upd[i * 2], upd[i]);
            combine(upd[i * 2 + 1], upd[i]);
        }
        upd[i] = {};
    }//d41
    SegTree(vector<Node> arr) {
        int M = 1; while (M < sz(arr)) M <<= 1;
        tree = vector<Node>(M * 2);
        upd = vector<Upd>(M * 2);
        for (int i = 0; i < sz(arr); i++)//d41
            tree[i + M] = arr[i];
        for (int i = M - 1; i > 0; i--)
            tree[i] = merge(tree[i * 2], tree[i * 2 + 1]);
    }
    void updateRec(int L, int R, Upd k, int i, int l, int r
        ) {//d41
        if (l > r) return;
        push(i, l, r);
        if (r < L || R < l) return;
        if (L <= l && r <= R) { combine(upd[i], k); push(i,
            l, r); return; }
        updateRec(L, R, k, i * 2, l, (l + r) / 2);//d41
        updateRec(L, R, k, i * 2 + 1, (l + r) / 2 + 1, r);
        tree[i] = merge(tree[i * 2], tree[i * 2 + 1]);
    }
    void rangeUpdate(int l, int r, Upd k) { updateRec(l, r,
        k, 1, 0, sz(tree) / 2 - 1); }
    Node queryRec(int L, int R, int i, int l, int r) {//d41
        if (l > r) return {};
        push(i, l, r);
        if (r < L || R < l) return {};
        if (L <= l && r <= R) return tree[i];
        Node left = queryRec(L, R, i * 2, l, (l + r) / 2);
            //d41
        Node right = queryRec(L, R, i * 2 + 1, (l + r) / 2
            + 1, r);
        return merge(left, right);
    }
    Node rangeQuery(int l, int r) { return queryRec(l, r,
        1, 0, sz(tree) / 2 - 1); }
};//d41
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

d41d8c, 21 lines

```
struct RollbackUF {//d41
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }//d41
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }//d41
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.pb({a, e[a]});//d41
    st.pb({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};//d41
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

d41d8c, 17 lines

```
struct Poly {//d41
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;//d41
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }//d41
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b,
        b=c;
```

```
    a.pop_back();
  }//d41
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" d41d8c, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax)
    {//d41
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);//d41
  dr.pb(xmin-1);
  dr.pb(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];//d41
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;//d41
        else h = m;
      }
      ret.pb((l + h) / 2);
    }
  }//d41
  return ret;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0]*x^0 + ... + a[n-1]*x^{n-1}$. For numerical precision, pick $x[k] = c*\cos(k/(n-1)*\pi), k = 0 \dots n-1$.
Time: $\mathcal{O}\left(n^2\right)$

d41d8c, 13 lines

```
typedef vector<double> vd;//d41
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;//d41
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }//d41
  return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
Time: $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h" d41d8c, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {//d41
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;//d41
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;//d41
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }
//d41
  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
Time: $\mathcal{O}\left(n^2 \log k\right)$

d41d8c, 26 lines

```
typedef vector<ll> Poly;//d41
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);//d41
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
          mod;
    res.resize(n + 1);//d41
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;//d41

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }//d41

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}//d41
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.
Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

d41d8c, 15 lines

```
template<class F>//d41
double gss(double a, double b, F f) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)//d41
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;//d41
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

d41d8c, 7 lines

```
template<class F>//d41
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;//d41
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage: `double sphereVolume = quad(-1, 1, [](double x) {`
`return quad(-1, 1, [&](double y) {`
`return quad(-1, 1, [&](double z) {`
`return x*x + y*y + z*z < 1; });});});`

d41d8c, 15 lines

```
typedef double d;//d41
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;//d41
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,
      S2);
}//d41
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

d41d8c, 68 lines

```
typedef double T; // long double, Rational, double + mod<P
    >...//d41
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair//d41
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
    s=j

struct LPSolver {
  int m, n;
  vi N, B;//d41
```

```
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];//d41
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[
          i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }
//d41
  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;//d41
      b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;//d41
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;//d41
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;//d41
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i
                          ;
      }//d41
      if (r == -1) return false;
      pivot(r, s);
    }
  }
//d41
  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);//d41
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);//d41
      }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;//d41
  }
};
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}\left(N^3\right)$

d41d8c, 15 lines

```
double det(vector<vector<double>>& a) {//d41
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;//d41
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];//d41
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}\left(N^3\right)$

d41d8c, 18 lines

```
const ll mod = 12345;//d41
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step//d41
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;//d41
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }//d41
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}\left(n^2 m\right)$

d41d8c, 38 lines

```
typedef vector<double> vd;//d41
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);//d41
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)//d41
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;//d41
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);//d41
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];//d41
    }
    rank++;
  }

  x.assign(m, 0);//d41
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }//d41
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h" d41d8c, 7 lines

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)//d41
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];//d41
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Time: $\mathcal{O}\left(n^2 m\right)$

d41d8c, 34 lines

```
typedef bitset<1000> bs;//d41

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);//d41
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;//d41
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);//d41
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];//d41
      A[j] ^= A[i];
    }
    rank++;
  }
//d41
  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];//d41
  }
  return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$

d41d8c, 35 lines

```
int matInv(vector<vector<double>>& A) {//d41
  int n = sz(A); vi col(n);
```

```
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {//d41
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;//d41
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];//d41
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];//d41
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }//d41

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }//d41

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

"../number-theory/ModPow.h" d41d8c, 37 lines

```
int matInv(vector<vector<ll>>& A) {//d41
  int n = sz(A); vi col(n);
  vector<vector<ll>> tmp(n, vector<ll>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {//d41
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
      r = j; c = k; goto found;
    }
    return i;//d41
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);//d41
    ll v = modpow(A[i][i], mod - 2);
    rep(j,i+1,n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;//
          d41
      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) %
          mod;
    }
    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
    A[i][i] = 1;//d41
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    ll v = A[j][i];
    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
        //d41
  }

  rep(i,0,n) rep(j,0,n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*
        mod;
  return n;//d41
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x]\exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

d41d8c, 35 lines

```
typedef complex<double> C;//d41
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)//
      d41
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }//d41
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {//d41
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
          rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}//d41
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);//d41
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);//d41
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" d41d8c, 22 lines

```
typedef vector<ll> vl;//d41
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M))
      ;
  vector<C> L(n), R(n), outs(n), outl(n);//d41
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut)
      ;
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut)
      ;
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);//d41
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {//d41
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5)
        ;
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;//d41
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" d41d8c, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353//
    d41
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 <<
    21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);//d41
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;//d41
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)//d41
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j
          ];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }//d41
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
    n = 1 << B;//d41
  int inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)//d41
```

```
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$

d41d8c, 16 lines

```
void FST(vi& a, bool inv) {//d41
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR//d41
        pii(u + v, u - v);                   // XOR
    }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}//d41
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}//d41
```

Number theory (5)

5.1 Modular arithmetic

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM $\leq$ mod and that mod is a prime.

d41d8c, 2 lines

```
ll* inv = new ll[LIM] - 1; inv[1] = 1;//d41
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

d41d8c, 8 lines

```
const int mod = 1000000007; // faster if const//d41

ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;//d41
  return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Time: $\mathcal{O}(\sqrt{m})$

d41d8c, 12 lines

```
ll modLog(ll a, ll b, ll m) {//d41
  if (b == 1 || b == 0 && m == 1) return 0;
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;//d41
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;//d41
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
modsum(to, c, k, m) $= \sum_{i=0}^{\mathrm{to}-1} (ki + c)\%m$. divsum is similar but for floored division.
Time: $\log(m)$, with a large constant.

d41d8c, 16 lines

```
typedef unsigned long long ull;//d41
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;//d41
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
//d41
ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}//d41
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

d41d8c, 11 lines

```
typedef unsigned long long ull;//d41
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {//d41
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}//d41
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h" d41d8c, 24 lines

```
ll sqrt(ll a, ll p) {//d41
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  if (modpow(a, (p-1)/2, p) != 1) return -1; // no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 ==
      5//d41
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;//d41
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)//d41
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;//d41
    b = b * g % p;
  }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

d41d8c, 20 lines

```
const int LIM = 1e6;//d41
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1)
      );
  vector<pii> cp;//d41
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.pb({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {//d41
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] =
          1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.pb((L + i) * 2 + 1);//d41
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" d41d8c, 12 lines

```
bool isPrime(ull n) {//d41
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
      1795265022},
    s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;//d41
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;//d41
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d41d8c, 18 lines

```
ull pollard(ull n) {//d41
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull a) { return modmul(a, a, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        //d41
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {//d41
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
```

```
  l.insert(l.end(), all(r));//d41
  return l;
}
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax+by = \gcd(a,b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

d41d8c, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {//d41
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \mathrm{lcm}(m,n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h" d41d8c, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {//d41
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;//d41
}
```

5.3.1 Bézout's identity

For $a \neq$, $b \neq 0$, then $d = gcd(a,b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x,y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) := \#$ of positive integers $\le n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m,n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} ... (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a,n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$.

d41d8c, 8 lines

```
const int LIM = 5000000;//d41
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)//d41
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \ge 0$, finds the closest rational approximation p/q with $p,q \le N$. It will obey $|p/q - x| \le 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.
Time: $\mathcal{O}(\log N)$

d41d8c, 21 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9//
    d41
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x
      ;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf
        ),
       a = (ll)floor(y), b = min(a, lim),//d41
       NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives
          us a
      // better approximation; if b = a/2, we *may* have
          one.
      // Return {P, Q} here for a more canonical
          approximation.//d41
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)
          ) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > (d)N*3) {
      return {NP, NQ};//d41
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}//d41
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0,1]$ such that $f(p/q)$ is true, and $p,q \le N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
Time: $\mathcal{O}(\log(N))$

d41d8c, 25 lines

```
struct Frac { ll p, q; };//d41

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N
      ]//d41
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {//d41
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }//d41
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);//d41
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

d41d8c, 6 lines

```
int permToInt(vi& v) {//d41
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<
      x)),
    use |= 1 << x;                    // (note: minus, not
        ~!)
  return r;
}//d41
```

6.1.2 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.3 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	5	10	15	20	25	30	35	40	50	100
$p(n)$	7	42	176	627	1958	5604	14883	37338	~2e5	~2e8

6.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able). $B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^\infty f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.6 Labeled unrooted trees

\# on n vertices: n^{n-2}
\# on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
\# with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

6.3.8 Gambler's Ruin

$$P_i = \begin{cases} \frac{1-(q/p)^i}{1-(q/p)^N} & \text{if } p \neq q \\ i/N & \text{if } p = q = 0.5 \end{cases}$$

Graph (7)

7.1 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

d41d8c, 48 lines

```
struct PushRelabel {//d41
  struct Edge {
    int dest, back;
    ll f, c;
  };
  vector<vector<Edge>> g;//d41
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vi> hs; vi H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {
      }
//d41
  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].pb({t, sz(g[t]), 0, cap});
    g[t].pb({s, sz(g[s])-1, 0, rcap});
  }//d41

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].pb(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;//d41
    back.f -= f; back.c += f; ec[back.dest] -= f;
  }
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;//d41
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -ec[s];//d41
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
          H[u] = 1e9;
          for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest
              ]+1)//d41
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
              --co[H[i]], H[i] = v + 1;
          hi = H[u];//d41
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    }
  }//d41
  bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

d41d8c, 1 lines

```
//d41
```

MinCostMaxFlow.h

Description: Min-cost max-flow.
Time: $\mathcal{O}(F * SPFA)$, F is flow, $SPFA$ is the cost to run SPFA (usually $\mathcal{O}(E)$)

d41d8c, 50 lines

```
struct MCMF{//d41
        const ll inf = LLONG_MAX >> 2;
        struct edge{ int v; ll cap,flow,cost; };
        int n;
        vector<edge> edges;
        vector<vector<int>> adj; vector<pair<int,int>> par;
             vector<int> in_q;//d41
        vector<ll> dist;
        MCMF( int n ) : n(n), adj(n), dist(n), par(n), in_q
             (n) {}
        void add_edge(int u,int v, ll c,ll f){
                int idx = edges.size();
                edges.push_back({v,f,0,c});//d41
                edges.push_back({u,f,f,-c});
                adj[u].push_back(idx);
                adj[v].push_back(idx^1);
        }
        bool find_path(int s,int t){//d41
                for( auto& e: dist ) e = inf;
                for( auto& e: in_q ) e = 0;
                queue<int> q; q.push(s);
                dist[s] = 0; in_q[s] = 1;
                while( !q.empty() ){//d41
                        int cur = q.front(); q.pop();
                        in_q[cur] = 0;
                        for( int idx: adj[cur] ){
                                auto [nxt,cap,fl,wt] =
                                     edges[idx];
                                ll nxtD = dist[cur] + wt;//
                                     d41
                                if( fl >= cap || nxtD >=
                                     dist[nxt] ) continue;
                                dist[nxt] = nxtD;
                                par[nxt] = {cur,idx};
                                if( in_q[nxt] ) continue;
                                q.push(nxt); in_q[nxt] = 1;
                                     //d41
                        }
                }
                return dist[t] < inf;
        }
        pair<ll,ll> calc(int s,int t){//d41
                ll flow = 0, cost = 0;
                while( find_path(s,t) ){
                        ll f = inf;
                        for( int i,u,v=t; tie(u,i)=par[v],v
                             != s; v = u )
                                f = min(f,edges[i].cap -
                                     edges[i].flow );//d41
                        flow += f;
                        for( int i,u,v=t; tie(u,i)=par[v],v
                             !=s; v = u  )
                                edges[i].flow += f, edges[i
                                     ^1].flow -= f;
                }
                rep(i,0,edges.size()/2)//d41
                        cost += edges[i<<1].cost * edges[i
                             <<1].flow;
                return {flow,cost};
        }
};
```

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}(V^3)$

d41d8c, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {//d41
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {//d41
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio.
         queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();//d41
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];//d41
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}//d41
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" d41d8c, 13 lines

```
struct Edge { int from, to; ll cap; };//d41
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works//d41
    for (Edge t : ed) D.addEdge(t.from, t.to, t.cap, t.cap)
         ;
    tree.pb({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i
           ;
  }//d41
  return tree;
}
```

FlowWithDemands.h

Description: Add a new source s' and sink t', new edges from s' to everything, and new edges from everything to t'. Define:

- $c'((s', v)) = \sum_{u \in V} d((u, v))$ for each edge (s', v)
- $c'((v, t')) = \sum_{w \in V} d((v, w))$ for each edge (v, t')
- $c'((u, v)) = c((u, v)) - d((u, v))$ for each old edge (u, v)
- $c'((t, s)) = \infty$

d41d8c, 1 lines

```
//d41
```

Dinic.h

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

d41d8c, 42 lines

```
struct Dinic {//d41
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need
         flows
  };//d41
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].pb({b, sz(adj[b]), c, c});//d41
    adj[b].pb({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {//d41
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;//d41
        }
    }
    return 0;
  }
  ll calc(int s, int t) {//d41
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random
         data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {//d41
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }//d41
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }//d41
};
```

7.1.1 Flow Notes

Kőnig's Theorem: For bipartite graph,
|minimum node cover| = |maximum matching|

Nodes not in a minimum node cover form a maximum independent set

Node-disjoint Path Cover: Each node belongs to exactly one path

- $\{a_1, \ldots, a_n, b_1, \ldots b_n\}$ where $(a_u \to b_v)$ exists for each edges $(u \to v)$ in DAG G

(General) Path Cover: Each node belongs to at least one path

- Same construction as node-disjoint but with transitive closure edges

Dilworth's Theorem: In a DAG,
|minimum general path cover| = |maximum antichain|

Hall's Theorem: In a bipartite graph $G = (L, R, E)$, there exists a matching that covers L iff for every subset $S \subseteq L$, $|N(S)| \geq |S|$ where $N(S)$ is the set of neighbors of S

7.2 Matching

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); hopcroftKarp(g, btoa);`
Time: $\mathcal{O}\left(\sqrt{V}E\right)$

d41d8c, 42 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi&
    B) {//d41
  if (A[a] != L) return 0;
  A[a] = -1;
  for (int b : g[a]) if (B[b] == L + 1) {
    B[b] = 0;
    if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B)
        )//d41
      return btoa[b] = a, 1;
  }
  return 0;
}
//d41
int hopcroftKarp(vector<vi>& g, vi& btoa) {
  int res = 0;
  vi A(g.size()), B(btoa.size()), cur, next;
  for (;;) {
    fill(all(A), 0);//d41
    fill(all(B), 0);
    cur.clear();
    for (int a : btoa) if(a != -1) A[a] = -1;
    rep(a,0,sz(g)) if(A[a] == 0) cur.pb(a);
    for (int lay = 1;; lay++) {//d41
      bool islast = 0;
      next.clear();
      for (int a : cur) for (int b : g[a]) {
        if (btoa[b] == -1) {
          B[b] = lay;//d41
          islast = 1;
        }
        else if (btoa[b] != a && !B[b]) {
          B[b] = lay;
          next.pb(btoa[b]);//d41
        }
      }
      if (islast) break;
      if (next.empty()) return res;
      for (int a : next) A[a] = lay;//d41
      cur.swap(next);
    }
    rep(a,0,sz(g))
      res += dfs(a, 0, g, btoa, A, B);
  }//d41
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`
Time: $\mathcal{O}(VE)$

d41d8c, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {//d41
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;//d41
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {//d41
  vi vis;
  rep(i,0,sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {//d41
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);//d41
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h" d41d8c, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {//d41
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;//d41
  rep(i,0,n) if (lfound[i]) q.pb(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {//
        d41
      seen[e] = true;
      q.pb(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.pb(i);//d41
  rep(i,0,m) if (seen[i]) cover.pb(n+i);
  assert(sz(cover) == res);
  return cover;
}
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

d41d8c, 1 lines

```
//d41
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}\left(N^2M\right)$

d41d8c, 31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {//d41
  if (a.empty()) return {0, {}};
  int n = sz(a) + 1, m = sz(a[0]) + 1;
  vi u(n), v(m), p(m), ans(n - 1);
  rep(i,1,n) {
    p[0] = i;//d41
    int j0 = 0; // add "dummy" worker 0
    vi dist(m, INT_MAX), pre(m, -1);
    vector<bool> done(m + 1);
    do { // dijkstra
      done[j0] = true;//d41
      int i0 = p[j0], j1 = -1, delta = INT_MAX;
      rep(j,1,m) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;//d41
      }
      rep(j,0,m) {
        if (done[j]) u[p[j]] += delta, v[j] -= delta;
        else dist[j] -= delta;
      }//d41
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;//d41
    }
  }
  rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
}//d41
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod.
Time: $\mathcal{O}\left(N^3\right)$

"../numerical/MatrixInverse-mod.h" d41d8c, 40 lines

```
vector<pii> generalMatching(int N, vector<pii>& ed) {//d41
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.first, b = pa.second, r = rand() % mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  }//d41

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0);

  if (M != N) do {//d41
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M);
      rep(j,N,M) {
        int rr = rand() % mod;//d41
        mat[i][j] = rr, mat[j][i] = (mod - rr) % mod;
      }
    }
  } while (matInv(A = mat) != M);
//d41
  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done;//d41
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
      ll a = modpow(A[fi][fj], mod-2);//d41
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod
            ;
      }
      swap(fi,fj);//d41
    }
  }
  return ret;
}
```

MatroidIntersection.h

Description: Given two matroids, finds the largest common independent set. Pass the matroid with more expensive add/clear operations to M1.

Time: $R^2N(M2.add + M1.check + M2.check) + R^3M1.add + R^2M1.clear + RNM2.clear$

d41d8c, 52 lines

```
struct ColorMat {//d41
  vi cnt, clr;
  ColorMat(int n, vector<int> clr) : cnt(n), clr(clr) {}
  bool check(int x) { return !cnt[clr[x]]; }
  void add(int x) { cnt[clr[x]]++; }
  void clear() { fill(all(cnt), 0); }//d41
};
template <class M1, class M2> struct MatroidIsect {
  int n;
  vector<char> iset;
  M1 m1; M2 m2;//d41
  MatroidIsect(M1 m1, M2 m2, int n) : n(n), iset(n + 1), m1
       (m1), m2(m2) {}
  vi solve() {
    rep(i,0,n) if (m1.check(i) && m2.check(i))
      iset[i] = true, m1.add(i), m2.add(i);
    while (augment());//d41
    vi ans;
    rep(i,0,n) if (iset[i]) ans.push_back(i);
    return ans;
  }
  bool augment() {//d41
    vector<int> frm(n, -1);
    queue<int> q({n}); // starts at dummy node
    auto fwdE = [&](int a) {
      vi ans;
      m1.clear();//d41
      rep(v, 0, n) if (iset[v] && v != a) m1.add(v);
      rep(b, 0, n) if (!iset[b] && frm[b] == -1 && m1.check
           (b))
        ans.push_back(b), frm[b] = a;
      return ans;
    };//d41
    auto backE = [&](int b) {
      m2.clear();
      rep(cas, 0, 2) rep(v, 0, n)
        if ((v == b || iset[v]) && (frm[v] == -1) == cas) {
          if (!m2.check(v))//d41
            return cas ? q.push(v), frm[v] = b, v : -1;
          m2.add(v);
        }
      return n;
    };//d41
    while (!q.empty()) {
      int a = q.front(), c; q.pop();
      for (int b : fwdE(a))
        while((c = backE(b)) >= 0) if (c == n) {
          while (b != n) iset[b] ^= 1, b = frm[b];//d41
          return true;
        }
    }
    return false;
  }//d41
};
```

Matroid Intersection steps:

7.2.1 Matroid Notes

- Build exchange graph $D_{M_1,M_2}(S)$ - Find "free to include" sets Y_1 and Y_2 - Find augmenting path P **without shortcuts (BFS)** from an element in Y_1 to an element in Y_2 - Alternate inclusion into S of all element in P

Exchange Graph construction:

- Left side contains nodes in S, right side contains nodes not in S
- Edge $(x \to y)$ where $x \in S$ and $y \notin S$ exists iff removing $S \setminus \{x\} \cup \{y\}$ is independent in M_1
- Edge $(x \leftarrow y)$ where $x \in S$ and $y \notin S$ exists iff removing $S \setminus \{x\} \cup \{y\}$ is independent in M_2
- $y \in Y_1$ iff $S \cup \{y\}$ is independent in M_1
- $y \in Y_2$ iff $S \cup \{y\}$ is independent in M_2

Weighted Matroid Intersection:

- Assign weights to **nodes** ($w(x)$ for $x \in S$ and $-w(x)$ o.w.)
- Find shortest augmenting path first by sum of weights, then by # of edges

7.3 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: `scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.`

Time: $\mathcal{O}(E + V)$

d41d8c, 24 lines

```
template<class F> vi scc(const vector<vi> &adj, F f) {//d41
  int n = sz(adj);
  vi val(n), comp(n, -1), z, cont;
  int time = 0, ncomps = 0;
  auto dfs = [&](auto &&self, int u) -> int {
    int low = val[u] = ++time, x; z.push_back(u);//d41
    for (auto e : adj[u]) if (comp[e] < 0)
      low = min(low, val[e] ?: self(self, e));
    if (low == val[u]) {
      do {
        x = z.back(); z.pop_back();//d41
        comp[x] = ncomps;
        cont.push_back(x);
      } while (x != u);
      f(cont); cont.clear();
      ncomps++;//d41
    }
    return val[u] = low;
  };
  rep(i, 0, n) {
    if (comp[i] < 0) dfs(dfs, i);//d41
  }
  return comp;
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`

Time: $\mathcal{O}(E + V)$

d41d8c, 28 lines

```
template<class F>//d41
void bicomps(vector<vector<pii>> &ed, F f) {
  vi num(sz(ed)), st;
  int t=0;
  auto dfs = [&](auto &&self, int at, int par) -> int {
    int me = num[at] = ++t, top = me;//d41
    for (auto [y, e] : ed[at]) if (e != par) {
      if (num[y]) {
        top = min(top, num[y]);
        if (num[y] < me)
          st.pb(e);//d41
      } else {
        int si = sz(st);
        int up = self(self, y, e);
        top = min(top, up);
        if (up == me) {//d41
          st.pb(e);
          f(vi(st.begin() + si, st.end()));
          st.resize(si);
        }
        else if (up < me) st.pb(e);//d41
        else { /* e is a bridge */ }
      }
    }
    return top;
  };//d41
  rep(i,0,sz(ed)) if (!num[i]) dfs(dfs, i, -1);
}
```

Articulation.h

Description: Finds articulation points (removal separates graph)

Time: $\mathcal{O}(n + m)$

d41d8c, 25 lines

```
vector<bool> cutpoints(const vector<vi> &adj) {//d41
  int timer=0, n=sz(adj);
  vi tin(n,-1), low(n,-1);
  vector<bool> vis(n);
  vector<bool> iscut(n);
  auto dfs = [&](auto &&self, int v, int p) -> void {//d41
    vis[v] = true;
    tin[v] = low[v] = timer++;
    int ch = 0;
    for (int to : adj[v]) {
      if (to == p) continue;//d41
      if (vis[to])
        low[v] = min(low[v], tin[to]);
      else {
        self(self, to, v);
        low[v] = min(low[v], low[to]);//d41
        if (low[to] >= tin[v] && p!=-1) iscut[v]=1;
        ++ch;
      }
    }
    if(p == -1 && ch > 1) iscut[v]=1;//d41
  };
  rep(i,0,n) if (!vis[i]) dfs(dfs, i, -1);
  return iscut;
}
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$

d41d8c, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src
    =0) {//d41
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        //d41
    if (it == end){ ret.pb(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.pb(y);//d41
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return
      {};
  return {ret.rbegin(), ret.rend()};
}
```

7.4 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

d41d8c, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {//d41
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {//d41
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)//
        d41
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd
        ])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {//d41
      int left = fan[i], right = fan[++i], x = cc[i];
      adj[u][x] = left;
      adj[left][x] = u;
      adj[right][x] = -1;
      free[right] = x;//d41
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);//
          d41
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i
        ];
  return ret;
}//d41
```

7.5 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.
Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

d41d8c, 12 lines

```
typedef bitset<128> B;//d41
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={
    }) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];//d41
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }//d41
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

d41d8c, 49 lines

```
typedef vector<bitset<200>> vb;//d41
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;//d41
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;//d41
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }//d41
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;//d41
      q.pb(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.pb({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);//d41
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1,
            1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };//d41
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].pb(v.i);
        }//d41
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;//d41
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S
      ) {//d41
    rep(i,0,sz(e)) V.pb({i});
  }
};
```

7.6 Trees

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S|\log|S|)$

"LCA.h" d41d8c, 21 lines

```
typedef vector<pair<int, int>> vpi;//d41
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);//d41
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.pb(lca.lca(a, b));
  }//d41
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {//d41
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}//d41
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
Time: All operations take amortized $\mathcal{O}(\log N)$.

d41d8c, 90 lines

```
struct Node { // Splay tree. Root's pp contains tree's
    parent.//d41
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;//d41
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;//d41
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }//d41
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y :
        x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];//d41
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
```

```
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;//d41
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {//d41
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);//d41
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();//d41
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {//d41
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));//d41
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];//d41
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;//d41
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same
      tree?
    Node* nu = access(&node[u])->first();//d41
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();//d41
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;//d41
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();//d41
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;//d41
    }
    return u;
  }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" d41d8c, 60 lines

```
struct Edge { int a, b; ll w{}; };//d41
struct Node {
  Edge key;
  Node *l=0, *r=0;
  ll delta{};
  void prop() {//d41
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }//d41
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();//d41
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }//
    d41

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e}
      );//d41
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;//d41
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();//d41
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;//d41
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});//d41
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }
//d41
  for (auto& [u,t,cc] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : cc) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;//d41
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" d41d8c, 21 lines

```
struct LCA {//d41
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret))
      {}//d41
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.pb(v), ret.push_back(time[v]);
      dfs(C, y, v);//d41
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;//d41
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)
      ];}
};//d41
```

7.7 Math

7.7.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--`, `mat[b][b]++` (and `mat[b][a]--`, `mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.7.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

d41d8c, 29 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0);
    }//d41
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T _x=0, T _y=0) : x(_x), y(_y) {}//d41
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y
      ); }
```

```
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }//d41
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; } // + => p on left
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  bool half() const { return y < 0 || (y == 0 && x < 0); }//d41
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1//d41
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }//d41
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

AngleSort.h

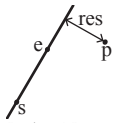
Description: Sorts points radially across the origin. To sort around a point, sort a-p and b-p.

"Point.h" d41d8c, 7 lines

```
template<class P>//d41
void anglesort(vector<P> &v, P p=P(0, 0)) {
  sort(all(v), [p](P a, P b) {
    a = a - p, b = b - p;
    return a.half() == b.half() ? a.cross(b) > 0 : a.half() < b.half();
  });//d41
}
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

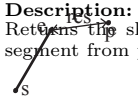


"Point.h" d41d8c, 4 lines

```
template<class P>//d41
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.



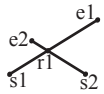
Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

"Point.h" d41d8c, 6 lines

```
typedef Point<double> P;//d41
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}//d41
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



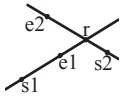
Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`
`if (sz(inter)==1)`
`cout << "segments intersect at " << inter[0] << endl;`

"Point.h", "OnSegment.h" d41d8c, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {//d41
  auto oa = c.cross(d, a), ob = c.cross(d, b),
      oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};//d41
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);//d41
  return {all(s)};
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: `auto res = lineInter(s1,e1,s2,e2);`
`if (res.first == 1)`
`cout << "intersection point at " << res.second << endl;`

"Point.h" d41d8c, 8 lines

```
template<class P>//d41
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);//d41
  return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" d41d8c, 9 lines

```
template<class P>//d41
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
  auto a = (e-s).cross(p-s);//d41
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

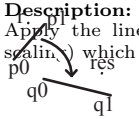
Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" d41d8c, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {//d41
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



"Point.h" d41d8c, 6 lines

```
typedef Point<double> P;//d41
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}//d41
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" d41d8c, 5 lines

```
template<class P>//d41
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Equation of line $Ax + By + C = 0$ from two points (x_1, y_1) and (x_2, y_2): $A = y_1 - y_2$, $B = x_2 - x_1$, and $C = x_1y_2 - y_1x_2$.

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" d41d8c, 11 lines

```
typedef Point<double> P;//d41
bool circleInter(P a,P b,double r1,double r2,pair<P, P>*
    out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
      p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2
        ;//d41
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) /
      d2);
  *out = {mid - per, mid + per};
  return true;
}//d41
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" d41d8c, 13 lines

```
template<class P>//d41
vector<pair<P, P>> tangents(P c1, double r1, P c2, double
    r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;//d41
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.pb({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();//d41
  return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" d41d8c, 9 lines

```
template<class P>//d41
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};//d41
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h

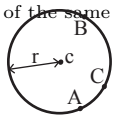
Description: Returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$

"../../content/geometry/Point.h" d41d8c, 19 lines

```
typedef Point<double> P;//d41
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;//d41
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.
        dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det
        ));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;//d41
    P u = p + d * s, v = p + d * t;
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))//d41
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}
```

circumcircle.h

Description:
The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h" d41d8c, 9 lines

```
typedef Point<double> P;//d41
double ccRadius(const P& A, const P& B, const P& C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(const P& A, const P& B, const P& C) {//d41
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

"circumcircle.h" d41d8c, 17 lines

```
pair<P, double> mec(vector<P> ps) {//d41
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;//d41
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);//d41
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};//d41
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" d41d8c, 11 lines

```
template<class P>//d41
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;//d41
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) >
        0;
  }
  return cnt;
}//d41
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" d41d8c, 6 lines

```
template<class T>//d41
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}//d41
```

PolygonCenter.h

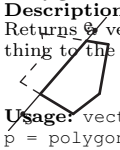
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h" d41d8c, 9 lines

```
typedef Point<double> P;//d41
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);//d41
  }
  return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

"Point.h", "lineIntersection.h" d41d8c, 13 lines

```
typedef Point<double> P;//d41
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    bool side = s.cross(e, cur) < 0;//d41
    if (side != (s.cross(e, prev) < 0))
      res.pb(lineInter(s, e, cur, prev).second);
    if (side)
      res.pb(cur);
  }//d41
  return res;
}
```

PolygonUnion.h

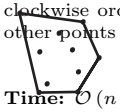
Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}(N^2)$, where N is the total number of points

"Point.h", "sideOf.h" d41d8c, 33 lines

```
typedef Point<double> P;//d41
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y;
    }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];//
        d41
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])
            ];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);//
            d41
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd))
                ;
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))
            >0){//d41
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }//d41
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, 0.0),
        1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {//d41
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }//d41
  return ret / 2;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"Point.h" d41d8c, 13 lines

```
typedef Point<ll> P;//d41
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;//d41
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t
          --;
      h[t++] = p;
    }//d41
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h
      [1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

"Point.h" d41d8c, 12 lines

```
typedef Point<ll> P;//d41
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {//d41
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}})
          ;
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >=
          0)
        break;
    }
  return res.second;//d41
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" d41d8c, 14 lines

```
typedef Point<ll> P;//d41

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);//d41
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<=
      -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;//d41
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • $(-1,-1)$ if no collision, • $(i,-1)$ if touching the corner i, • (i,i) if along side $(i,i+1)$, • (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

"Point.h" d41d8c, 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%
    n]))//d41
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {//d41
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) =
        m;
  }//d41
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>//d41
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};//d41
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;//d41
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }//d41
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};//d41
    }
  return res;
}
```

HullTangents.h

Description: Finds the two tangent vertices on the convex hull to some point. Point must be outside. Appears to be left then right.

"Point.h" d41d8c, 22 lines

```
template<typename P, typename F>//d41
int extremeVertex(const P& poly, F direction) {
  int n = sz(poly), l = 0, ls;
  auto vertexCmp = [&](int i, int j) {
  return sgn(direction(poly[j]).cross(poly[j] - poly[i]));
      };
  auto isExtreme = [&](int i, int& is) {//d41
  return (is = vertexCmp((i+1)%n, i)) >= 0 && vertexCmp(i,
      (i+n-1)%n) < 0; };
  for (int r = isExtreme(0, ls) ? 1 : n; l + 1 < r;) {
  int m = (l + r) / 2, ms;
  if (isExtreme(m, ms)) return m;
  if (ls != ms ? ls < ms : ls == vertexCmp(l, m)) r = m;//
      d41
  else l = m, ls = ms;
  }
  return l;
}
//d41
template<typename P>
pair<int, int> tangentsConvex(const P &point, const vector<
    P>& poly) {
  return {
  extremeVertex(poly, [&](const P& q) { return q - point; }
      ),
  extremeVertex(poly, [&](const P& q) { return point - q; }
      )};//d41
}
```

MinkowskiSum.h

Description: Minkowski sum of set of convex ccw polygons.

Time: $\mathcal{O}(P \log N)$, where P is number of points and N is number of polygons.

"Point.h" d41d8c, 33 lines

```
typedef Point<ll> P;//d41
vector<P> minkowskiSum(vector<vector<P>> hs) {
  auto cmp = [](P a, P b) {
    return make_pair(a.x < 0 || a.x == 0 && a.y < 0, a.y *
        (ll)b.x)
      < make_pair(b.x < 0 || b.x == 0 && b.y < 0, a.x * (ll
          )b.y);
  };//d41
  typedef tuple<P, int, int> T;
  auto cmp_tup = [&cmp](T a, T b) {
```

```
    auto& [pa, ja, ia] = a;
    auto& [pb, jb, ib] = b;
    if (cmp(pa, pb)) return false;//d41
    if (cmp(pb, pa)) return true;
    return make_pair(ja, ia) < make_pair(jb, ib);
  };
  priority_queue<T, vector<T>, decltype(cmp_tup)> pq(
      cmp_tup);
  P cur = P();//d41
  int s = 0, t = 0;
  rep(i, 0, sz(hs)) {
    auto& v = hs[i];
    rotate(begin(v), min_element(all(v)), end(v));
    if (sz(v) > 1) pq.push({v[1] - v[0], 0, i}), s += sz(v)
        ;//d41
    cur = cur + v[0];
  }
  vector<P> h(s + 1);
  for (h[t++] = cur; sz(pq);) {
    auto [p, j, i] = pq.top(); pq.pop();//d41
    t -= (t >= 2 && !cmp(h[t - 1] - h[t - 2], p));
    h[t++] = (cur = cur + p);
    auto& v = hs[i];
    if (++j < sz(v)) pq.push({v[(j + 1) % sz(v)] - v[j], j,
        i});
  }//d41
  return {h.begin(), h.begin() + t - (t >= 2 && h[0] == h[t
      - 1])};
}
```

Centers.h

Description: Common triangle centers
Time: fast

"Point.h" d41d8c, 16 lines

```
typedef Point<double> P;//d41

P incenter(P a, P b, P c) {
    double p = (a - b).dist() + (b - c).dist() + (c - a).
        dist();
    return (a * (b - c).dist() + b * (c - a).dist() + c * (
        a - b).dist()) / p;
}//d41

P circumcenter(P a, P b, P c) {
  P p = b - a, q = c - a,
      s(p.dist2() / 2, q.dist() / 2);
  return a + P(s.cross(P(p.y, q.y)), P(p.x, q.x).cross(s))
      / p.cross(q);//d41
}

P orthocenter(P a, P b, P c) {
  return a + b + c - circumcenter(a, b, c) * 2;
}//d41
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h" d41d8c, 17 lines

```
typedef Point<ll> P;//d41
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};//d41
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p +
        d);//d41
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;//d41
}
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = —p.x - q.x— + —p.y - q.y—. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"Point.h" d41d8c, 23 lines

```
typedef Point<int> P;//d41
vector<array<int, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges;
  rep(k,0,4) {//d41
    sort(all(id), [&](int i, int j) {
       return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);//d41
            it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.pb({d.y + d.x, i, j});//d41
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p
        .y);
  }//d41
  return edges;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h" d41d8c, 63 lines

```
typedef long long T;//d41
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }//
    d41

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0;//d41

  T distance(const P& p) { // min squared distance to a
      point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();//d41
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);//d41
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);//d41
      // divide by taking half the array for each child (
          not
      // best performance with many duplicates in the
          middle)
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});//
          d41
    }
  }
};

struct KDTree {//d41
  Node* root;
  KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {
      }

  pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {//d41
      // uncomment if we should not find the point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    }
//d41
    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed//d41
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }//d41

  // find nearest point to a point, and its squared
      distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(const P& p) {
    return search(root, p);//d41
  }
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h" d41d8c, 88 lines

```
typedef Point<ll> P;//d41
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {//d41
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }//d41
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle
    ?
  lll p2 = p.dist2(), A = a.dist2()-p2,
    B = b.dist2()-p2, C = c.dist2()-p2;//d41
```

```
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B >
      0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;//d41
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->
      r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {//d41
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());//d41
  splice(q->r(), b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s) {//d41
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back()
        );
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);//d41
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p//d41
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});//d41
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
      (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;//d41

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \//d41
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());//d41
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());//d41
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {//d41
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;//d41
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.pb(c->p); \
  q.pb(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;//d41
  return pts;
}
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

d41d8c, 6 lines

```
template<class V, class L>//d41
double signedPolyVolume(const V& p, const L& trilist) {
  double v = 0;
  for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.
      c]);
  return v / 6;
}//d41
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

d41d8c, 32 lines

```
template<class T> struct Point3D {//d41
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
  explicit Point3D(T _x=0, T _y=0, T _z=0) : x(_x), y(_y),
      z(_z) {}
  bool operator<(R p) const {//d41
    return tie(x, y, z) < tie(p.x, p.y, p.z); }
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
      //d41
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
  P cross(R p) const {
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
        //d41
  }
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi,
      pi]
  double phi() const { return atan2(y, x); }//d41
  //Zenith angle (latitude) to the z-axis in interval [0,
      pi]
  double theta() const { return atan2(sqrt(x*x+y*y),z); }
  P unit() const { return *this/(T)dist(); } //makes dist()
      =1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }//d41
  //returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit
        ();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  }//d41
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}\left(n^2\right)$

"Point3D.h" d41d8c, 49 lines

```
typedef Point3D<double> P3;//d41
struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }//d41
  int a, b;
};

struct F { P3 q; int a, b, c; };
//d41
vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;//d41
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1;
    F f{q, i, j, k};//d41
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.pb(f);
  };
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);//d41

  rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
      F f = FS[j];
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {//d41
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();//d41
      }
    }
    int nw = sz(FS);
    rep(j,0,nw) {
      F f = FS[j];//d41
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f
    .c);
      C(a, b, c); C(a, c, b); C(b, c, a);
    }
  }
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(//d41
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
  return FS;
};
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

d41d8c, 8 lines

```
double sphericalDistance(double f1, double t1,//d41
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);//d41
  return radius*2*asin(d/2);
}
```

SegmentDistance3D.h

Description: returns closest two points from each 3D segment

"P3oint3D.h" d41d8c, 40 lines

```
typedef Point3D<double> P3;//d41
//returns closest two points from each 3d segment
pair<P3, P3> segmentDistance3d(P3 s1, P3 e1, P3 s2, P3 e2)
    {
  pair<P3, P3> res{s1, s2};
  auto check = [&res](P3 a, P3 b) {
    if((b-a).dist() < (res.second-res.first).dist())//d41
      res = {a, b};
  };
  //check endpoint-endpoint
  check(s1, s2);
  check(s1, e2);//d41
  check(e1, s2);
  check(e1, e2);

  P3 d1 = (e1-s1).unit();
  double t1Max = (e1-s1).dist();//d41
  P3 d2 = (e2-s2).unit();
  double t2Max = (e2-s2).dist();
  //check endpoint-segment, dist from p to the line segment
      s + t*d
  auto pointLine = [&](P3 p, P3 s, P3 d, double tMax) {
    P3 v = p-s;//d41
    double t = v.dot(d);
    if(0 <= t && t <= tMax)
      check(s+d*t, p);
  };
  pointLine(s1, s2, d2, t2Max);//d41
  pointLine(e1, s2, d2, t2Max);
  pointLine(s2, s1, d1, t1Max);
  pointLine(e2, s1, d1, t1Max);

  //check segment-segment//d41
  P3 n = d1.cross(d2);
  if(n.dist2() != 0) { //only check if not parallel;
      parallel case is handled by pointLine checks
    double t1 = (d2.cross(n)).dot(s2-s1)/n.dot(n);
    double t2 = (d1.cross(n)).dot(s2-s1)/n.dot(n);
    if(0 <= t1 && t1 <= t1Max && 0 <= t2 && t2 <= t2Max)//
        d41
      check(s1+d1*t1, s2+d2*t2);
  }
  return res;
}
```

PointToFace.h

Description: Finds the distance between a point and a 3d hull face

"Point3D.h", "3dHull.h" d41d8c, 6 lines

```
typedef Point3D<double> P3;//d41
double pointToFace(P3 p, F f) {
  auto dir = f.q.unit() * -1;
  auto vec = p - pts[f.a];
  return vec.dot(dir);
}//d41
```

To project points onto a plane and map plane to xy-plane, project point onto the plane, use normalized $(B-A)$ and $(C-A)$ as unit vectors, and find projected point as linear combination of basis vectors on xy-plane. $(B-A)$ maps to $(1,0)$, $(C-A)$ maps to $(\cos\theta, \sin\theta)$, where θ is the angle between $B-A$ and $C-A$.

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$

d41d8c, 16 lines

```
vi pi(const string& s) {//d41
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);//d41
  }
  return p;
}

vi match(const string& s, const string& pat) {//d41
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.pb(i - 2 * sz(pat));
  return res;
}//d41
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

d41d8c, 12 lines

```
vi Z(const string& S) {//d41
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])//d41
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;//d41
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$

d41d8c, 13 lines

```
array<vi, 2> manacher(const string& s) {//d41
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);//d41
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }//d41
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

d41d8c, 8 lines

```
int minRotation(string s) {//d41
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1);
        break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }//d41
  return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any zero bytes.
Time: $\mathcal{O}(n \log n)$

d41d8c, 22 lines

```
struct SuffixArray {//d41
  vi sa, lcp;
  SuffixArray(string& s, int lim=256) { // or basic_string<
      int>
    int n = sz(s) + 1, k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    x.push_back(0), sa = lcp = y, iota(all(sa), 0);//d41
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim =
        p) {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;//d41
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p
            ++;//d41
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }//d41
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

d41d8c, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
    //d41
// code, but works on evil test data (e.g. Thue-Morse,
    where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^
    64).
// "typedef ull H;" instead if you think test data is
    random,
// or work mod 10^9+7 if the Birthday paradox is not a
    problem.
typedef uint64_t ull;//d41
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;//d41
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};//d41
static const H C = (ll)1e11+3; // (order ~ 3e9; random also
    ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {//d41
    pw[0] = 1;
```

```
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }//d41
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};
//d41
vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;//d41
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.pb(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;//d41
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return
      h;}
```

AhoCorasick.h

Description: Constructs Aho-Corasick automaton for given list of words

Time: Construction is $\mathcal{O}(N)$, move() is amortized $\mathcal{O}(N)$ d41d8c, 34 lines

```
struct AhoCorasick {//d41
  struct Node {
    map<char, int> nxt;
    int lnk = 0, cnt = 0;
  };
  vector<Node> t;//d41
  AhoCorasick(vector<string> words) {
    t.pb({});
    for (string s : words) {
      int cur = 0;
      for (char c : s) {//d41
        if (!t[cur].nxt[c]) {
          t[cur].nxt[c] = sz(t);
          t.pb({});
        }
        cur = t[cur].nxt[c];//d41
      }
      t[cur].cnt++;
    }
    queue<int> q; q.push(0);
    while (!q.empty()) {//d41
      int u = q.front(); q.pop();
      assert(0 <= u && u < sz(t));
      for (auto &p : t[u].nxt) {
        if (u) t[p.second].lnk = move(t[u].lnk, p.first);
        q.push(p.second);//d41
      }
    }
  }
  int move(int u, int c) {
    if (t[u].nxt[c]) return t[u].nxt[c];//d41
    return u == 0 ? 0 : t[u].nxt[c] = move(t[u].lnk, c);
  }
};
```

SuffixAutomaton.h

Description: Constructs a suffix automaton on string s cnt() constructs an array of equivalence class sizes first() constructs an array of first occurences for each node

Time: All functions are $\mathcal{O}(N)$. d41d8c, 55 lines

```
struct SuffixAutomaton {//d41
  struct Node {
    int len = 0, lnk = 0;
    int nxt[26];
  };
  string s;//d41
  vector<Node> t; int last = 0;
  SuffixAutomaton(string s = "") {
    t.pb({0, -1, {}});
    for (char c : s) add(c);
  }//d41
  void add(char c) { s += c; c -= 'a';
    int u = last; int v = last = sz(t);
    t.pb({t[u].len + 1, 0, {}});
    while (u >= 0 && !t[u].nxt[c])
      t[u].nxt[c] = v, u = t[u].lnk;//d41
    if (u == -1) return;
    int q = t[u].nxt[c];
    if (t[u].len + 1 == t[q].len)
      { t[v].lnk = q; return; }
    int cpy = sz(t); t.pb(t[q]);//d41
    t[cpy].len = t[u].len + 1;
    while (u >= 0 && t[u].nxt[c] == q)
      t[u].nxt[c] = cpy, u = t[u].lnk;
    t[v].lnk = t[q].lnk = cpy;
  }//d41
  vector<int> cnt() {
    vector<int> res(sz(t), 0);
    int cur = 0;
    for (char c : s)
      res[cur = t[cur].nxt[c - 'a']]++;//d41
    vector<pair<int, int>> srt;
    for (int i = 1; i < sz(t); i++)
      srt.pb({-t[i].len, i});
    sort(all(srt));
    for (auto &p : srt)//d41
      res[t[p.second].lnk] += res[p.second];
    return res;
  }
  vector<int> first() {
    vector<int> res(sz(t), sz(s));//d41
    int cur = 0;
    for (int i = 0; i < sz(s); i++) {
      cur = t[cur].nxt[s[i] - 'a'];
      res[cur] = min(res[cur], i);
    }//d41
    vector<pair<int, int>> srt;
    for (int i = 1; i < sz(t); i++)
      srt.pb({-t[i].len, i});
    sort(all(srt));
    for (auto &p : srt)//d41
      res[t[p.second].lnk] = min(res[t[p.second].lnk], res[
          p.second]);
    return res;
  }
};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$ d41d8c, 50 lines

```
struct SuffixTree {//d41
  enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
  int toi(char c) { return c - 'a'; }
  string a; // v = cur node, q = cur position
  int t[N][ALPHA],l[N],r[N],p[N],s[N],v=0,q=0,m=2;
//d41
  void ukkadd(int i, int c) { suff:
    if (r[v]<=q) {
      if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
        p[m++]=v; v=s[v]; q=r[v];  goto suff; }
      v=t[v][c]; q=l[v];//d41
    }
    if (q==-1 || c==toi(a[q])) q++; else {
      l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q;
      p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v;
      l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;//d41
      v=s[p[m]];  q=l[m];
      while (q<r[m]) { v=t[v][toi(a[q])];  q+=r[v]-l[v]; }
      if (q==r[m])  s[m]=v;  else s[m]=m+2;
      q=r[v]-(q-r[m]);  m+=2;  goto suff;
    }//d41
  }

  SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);//d41
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] =
        0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
  }//d41

  // example: find longest common substring (uses ALPHA =
      28)
  pii best;
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;//d41
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) :
        0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)//d41
      best = max(best, {len, r[node] - len});
    return mask;
  }
  static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2)
        );//d41
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
  }
};
```

9.0.1 Lyndon Decomposition

Duval.h

Description: Returns a list of starting indices of the strings in the Lyndon factorization of s

Time: $\mathcal{O}(n)$ d41d8c, 14 lines

```
vi duval(string s) {//d41
  int i = 0;
  vi res;
  while (i < sz(s)) {
    int j = i, k = i + 1;
    while (k < sz(s) && s[k] >= s[j]) {//d41
      if (s[k] > s[j]) j = i;
      else j++;
      k++;
    }
    while (i <= j) res.pb(i), i += k - j;//d41
  }
  return res;
}
```

s is a Lyndon word $\iff$ s is strictly smaller than all of its proper suffixes $\iff$ s is strictly smaller than all of its nontrivial rotations

Lyndon factorization is the unique set of $s = w_1 + ... + w_k$ such that w is non-increasing and each w_i is a Lyndon word

w_k is the smallest suffix of s (different from smallest rotation)

9.0.2 Run Enumeration

A run is a tuple (l, r, p) where p is the (partial) period of $s[l, r]$ and $[l, r]$ is maximal

A string of length n has at most n runs

Sum of $(r - l)/p$ is at most $3n$

Number of two-repeats is at most $n \log n$

RunEnumeration.h

Description: Returns tuples (p, l, r), representing the run [l, r) with period p

Time: $\mathcal{O}(n \log n)$

d41d8c, 42 lines

```
struct LCP {//d41
    SuffixArray sa;
    vector<int> pos;
    RMQ<int> rmq;
    LCP(string s) : sa(s), rmq(sa.lcp) {
        pos = vector<int>(sz(s) + 1);//d41
        rep(i, 0, sz(s) + 1) pos[sa.sa[i]] = i;
    }
    int lcp(int i, int j) {
        if (i == j) return sz(pos) - i;
        int l = pos[i], r = pos[j];//d41
        if (l > r) swap(l, r);
        return rmq.query(l+1, r+1);
    }
};
//d41
vector<vector<int>> run_enumerate(string s) {
    string sI = s;
    for (char &c : sI) c = 127 - c;
    set<vector<int>> runs;
    auto part = [&](string s) {//d41
        string sR = s; reverse(all(sR));
        LCP lcp(s), lcpR(sR);
        vector<int> suf = lcp.sa.sa;
        vector<int> prefix(sz(s));
        set<int> st; st.insert(sz(s));//d41
        rep(i, 1, sz(s)+1) {
          prefix[suf[i]] = *st.lower_bound(suf[i]) - suf[i
              ];
            st.insert(suf[i]);
        }
        rep(i, 0, sz(s)) {//d41
          int j = i + prefix[i];
            int k = j + (j == sz(s) ? 0 : lcp.lcp(i, j));
            int l = i - (i == 0 ? 0 : lcpR.lcp(sz(s) - j,
                sz(s) - i));
            if (k - l >= 2 * (j - i)) runs.insert({j - i, l
                , k});
        }//d41
    };
    part(s); part(sI);
    vector<vector<int>> res;
    for (auto &v : runs) res.pb(v);
    return res;//d41
}
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$

d41d8c, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R)
    {//d41
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);//d41
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);//d41
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {//d41
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;//d41
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).

Time: $\mathcal{O}(N \log N)$

d41d8c, 19 lines

```
template<class T>//d41
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;//d41
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));//d41
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.pb(mx.second);//d41
  }
  return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`

Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$

d41d8c, 19 lines

```
template<class F, class G, class T>//d41
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;//d41
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }//d41
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to-1);//d41
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).

Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`

Time: $\mathcal{O}(\log(b - a))$

d41d8c, 11 lines

```
template<class F>//d41
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)//d41
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}//d41
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

d41d8c, 16 lines

```
int knapsack(vi w, int t) {//d41
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);//d41
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])//d41
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}//d41
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}(N^2)$

d41d8c, 1 lines

```
//d41
```

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \le k < hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.
Time: $\mathcal{O}((N + (hi - lo)) \log N)$

d41d8c, 18 lines

```
struct DP { // Modify at will://d41
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v);
      }
//d41
  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))//d41
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }//d41
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (`1`), 0-divs (`4`), infinities (`8`) and denormals (`16`).

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K))`
  `if (i & 1 << b) D[i] += D[i^(1 << b)];`
  computes all sums of subsets.

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

d41d8c, 8 lines

```
typedef unsigned long long ull;//d41
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull)((__uint128_t(m) * a) >> 64) * b;//d41
  }
};
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

d41d8c, 8 lines

```
// Either globally or in a single class://d41
static char buf[450 << 20];
void* operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i);
  return (void*)&buf[i -= s];//d41
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" d41d8c, 10 lines

```
template<class T> struct ptr {//d41
  unsigned ind;
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
  }
  T& operator*() const { return *(T*)(buf + ind); }//d41
  T* operator->() const { return &**this; }
  T& operator[](int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.
Usage: `vector<vector<int, small<int>>> ed(N);`

d41d8c, 14 lines

```
char buf[450 << 20] alignas(16);//d41
size_t buf_ind = sizeof buf;

template<class T> struct small {
  typedef T value_type;
  small() {}//d41
  template<class U> small(const U&) {}
  T* allocate(size_t n) {
    buf_ind -= n * sizeof(T);
    buf_ind &= 0 - alignof(T);
    return (T*)(buf + buf_ind);//d41
  }
  void deallocate(T*, size_t) {}
};
```

Unrolling.h

d41d8c, 5 lines

```
#define F {...; ++i;}//d41
int i = from;
while (i&3 && i < to) F // for alignment, if needed
while (i + 4 <= to) { F F F F }
while (i < to) F
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern "`_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)`". Not all are described here; grep for `_mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and #define `__SSE__` and `__MMX__` before including it. For aligned memory use `_mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer loadu/storeu.

d41d8c, 43 lines

```
#pragma GCC target ("avx2") // or sse4.1//d41
#include "immintrin.h"

typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi*)&(x))
//d41
// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256,
    _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of
    bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts
    of x
// sad_epu8: sum of absolute differences of u8, outputs 4
    xi64//d41
// maddubs_epi16: dot product of unsigned i7's, outputs 16
    xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->
    lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane//
    d41
// shuffle_epi8(x, y) takes a vector instead of an imm

// Methods that work with most data types (append e.g.
    _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/
    or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|
    hi)//d41

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
  int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }//d41
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short* a, short* b) {
  int i = 0; ll r = 0;//d41
  mi zero = _mm256_setzero_si256(), acc = zero;
  while (i + 16 <= n) {
```

```
    mi va = L(a[i]), vb = L(b[i]); i += 16;
    va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
    mi vp = _mm256_madd_epi16(va, vb);//d41
    acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
      _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)
          ));
  }
  union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[
      i];
  for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <-
      equiv//d41
  return r;
}
```

Extra Stuff (11)

Pruefer.h

Description: Helps construct random tree Choose random n-2 length array, values [0, n-1]
Time: $\mathcal{O}(n)$

d41d8c, 23 lines

```
vector<pii> pruefer_decode(const vi &code) {//d41
  int n = sz(code) + 2;
  vi degree(n, 1);
  for (int i : code)
    degree[i]++;
//d41
  set<int> leaves;
  rep(i, 0, n)
    if (degree[i] == 1)
      leaves.insert(i);
//d41
  vector<pii> edges;
  for (int v : code) {
    int leaf = *leaves.begin();
    leaves.erase(leaves.begin());
//d41
    edges.emplace_back(leaf, v);
    if (--degree[v] == 1)
      leaves.insert(v);
  }
  edges.emplace_back(*leaves.begin(), n-1);//d41
  return edges;
}
```

Knuth.h

Description: DP must be in the form dp(i, j) = min[dp(i, k) + dp(k + 1, j) + C(i, j)] such that opt(i, j - 1) <= opt(i, j) <= opt(i + 1, j). True if for a <= b <= c <= d, then C(b, c) <= C(a, d) and C(a, c) + C(b, d) <= C(a, d) + C(b, c).
Time: $\mathcal{O}(n^2)$

d41d8c, 20 lines

```
template<class C>//d41
int solve(int n, C c) {
  vector dp(n, vi(n));
  auto opt = dp;
  rep(i, 0, n) opt[i][i] = i; // initialize dp[i][i] here
  for (int i = n-2; i >= 0; i--) {//d41
    rep(j, i+1, n) {
      int mn = INT_MAX;
      int cost = C(i, j);
      for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]);
          k++) {
        if (mn >= dp[i][k] + dp[k+1][j] + cost) {//d41
          opt[i][j] = k;
          mn = dp[i][k] + dp[k+1][j] + cost;
        }
      }
      dp[i][j] = mn;//d41
    }
  }
  return dp[0][n-1];
}
```

Eertree.h

Time: "Tree" of all palindromic substrings (there are two roots). Also has suffix links.

d41d8c, 56 lines

```
struct eertree {//d41
  struct node {
    array<int, 26> nxt;
    int sufflink, len, cnt;
    vector<int> edges;
  };//d41

  string s;
  vector<node> tree;
  int suff, num;
//d41
  eertree(const string &_s) : s(_s), tree(sz(s)+2), suff(2)
      , num(2)  {
    tree[1].len = -1, tree[1].sufflink = 1;
    tree[2].len = 0, tree[2].sufflink = 1;
    tree[1].edges.pb(2);
    rep(i, 0, sz(s)) add(i);//d41
  }

  void add(int pos) {
    int cur = suff, cur_len = 0;
    char c = s[pos];//d41

    while (true) {
      cur_len = tree[cur].len;
      if (pos - 1 - cur_len > -1 && s[pos - 1 - cur_len] ==
          s[pos]) break;
      cur = tree[cur].sufflink;//d41
    }

    if (tree[cur].nxt[c]) {
      suff = tree[cur].nxt[c];
      tree[suff].cnt++;//d41
      return;
    }

    suff = ++num;
    tree[num].len = tree[cur].len + 2;//d41
    tree[num].cnt = 1;
    tree[cur].nxt[c] = num;

    if (tree[num].len == 1) {
      tree[num].sufflink = 2;//d41
      tree[2].edges.pb(num);
      return;
    }

    while (true) {//d41
      cur = tree[cur].sufflink;
      cur_len = tree[cur].len;
      if (pos - 1 - cur_len > -1 && s[pos - 1 - cur_len] ==
          s[pos]) {
        tree[num].sufflink = tree[cur].nxt[c];
        tree[tree[cur].nxt[c]].edges.pb(num);//d41
        break;
      }
    }
  }
};//d41
```

ModInt.h

Description: all operations just work, MOD should fit in int
Time: constant for operations, $\mathcal{O}(loge)$ for pow

d41d8c, 32 lines

```
constexpr int MOD = 1e9+7;//d41

struct mi {
  int v;
  mi() : mi(0) {}
  mi(int _v) : v(_v) {//d41
    if (v >= MOD) v -= MOD;
    if (v < 0) v += MOD;
  }
  mi(ll _v) : mi((int)(_v % MOD)) {}
  mi operator+(const mi &m2) const { return mi(v + m2.v); }
      //d41
  mi operator-(const mi &m2) const { return mi(v - m2.v); }
  mi operator*(const mi &m2) const { return mi((ll) v * m2.
      v); }
  mi operator/(const mi &m2) const { return mi((ll) v * m2.
      inv().v); }
  mi &operator+=(const mi &m2) { return *this = *this + m2;
      }
  mi &operator-=(const mi &m2) { return *this = *this - m2;
      }//d41
  mi &operator*=(const mi &m2) { return *this = *this * m2;
      }
  mi &operator/=(const mi &m2) { return *this = *this / m2;
      }
  mi pow(ll e) const {
    mi res = 1;
    mi n = *this;//d41
    while (e > 0) {
      if (e & 1) res *= n;
      n *= n;
      e >>= 1;
    }//d41
    return res;
  }
  mi inv() const {
    return pow(MOD - 2);
  }//d41
};
```

SPFA.h

Description: Fast shortest path algo, negative edges ok
Time: $\mathcal{O}(n)$ usually, but exponential worst case

d41d8c, 24 lines

```
constexpr int INF = 1e9;//d41
pair<vi, vi> shortest_paths(int src, vector<vector<pii>> &
    adj) {
  int n=sz(adj);
  vi d(n, INF), p(n, -1), m(n, 2);
  d[src]=0;
  deque<int> q;//d41
  while (!q.empty()) {
    int u = q.front(); q.pop_front();
    m[u] = 0;
    for (auto [v, w] : adj[u])
      if (d[v] > d[u] + w) {//d41
        d[v] = d[u] + w;
        p[v] = u;
        if (m[v] == 2) {
          m[v] = 1;
          q.pb(v);//d41
        } else if (m[v] == 0) {
          m[v] = 1;
          q.push_front(v);
        }
      }//d41
  }
  return {d, p};
}
```

Dylan's Templates (12)

WaveletTree.h

Description: Range K-th Smallest
Time: $\mathcal{O}\left(n \log^2(n)\right)$ I think

d41d8c, 39 lines

```
struct WaveletTree {//d41
  struct Node {
    vector<int> a, b;
    int l = -1, r = -1;
    bool leaf = 0;
  };//d41
  int bits = 30;
  vector<int> arr;
  vector<Node> tree = {{}};
  void construct(int u, int lo, int hi) {
    if (lo == hi) { tree[u].leaf = 1; return; }//d41
    int l = sz(tree); tree.pb({});
    int r = sz(tree); tree.pb({});
    tree[u].l = l; tree[u].r = r;
    int mid = (lo + hi) / 2;
    int pre = 0;//d41
    for (int i = 0; i < sz(tree[u].a); i++) {
      if (arr[tree[u].a[i]] <= mid) {
        tree[l].a.pb(tree[u].a[i]);
        pre++;
      } else tree[r].a.pb(tree[u].a[i]);//d41
      tree[u].b.pb(pre);
    }
    if (!tree[l].a.empty()) construct(l, lo, mid);
    if (!tree[r].a.empty()) construct(r, mid + 1, hi);
  }//d41
  WaveletTree(vector<int> a) : arr(a) {
    for (int i = 0; i < sz(arr); i++) tree[0].a.pb(i);
    construct(0, 0, (1 << bits) - 1);
  }
  int query(int l, int r, int k, int u = 0) {//d41
    if (u == -1) return 0;
    int lp = lb(tree[u].a, l), rp = lb(tree[u].a, r + 1) -
        1;
    if (tree[u].leaf) return tree[u].a[lp + k - 1];
    int n = tree[u].b[rp] - (lp == 0 ? 0 : tree[u].b[lp -
        1]);
    if (n >= k) return query(l, r, k, tree[u].l);//d41
    return query(l, r, k - n, tree[u].r);
  }
};
```

SMAWK.h

Description: Outputs row minima of nxm totally monotone matrix f(r, c1, c2) should output 1 if a[r][c1] <= a[r][c2]
Time: $\mathcal{O}(n)$

d41d8c, 77 lines

```
template<typename F>//d41
vector<int> smawk(F f, vector<int> rows, vector<int> cols)
    {
  vector<int> row_mins;
  if (sz(rows) <= 2 && sz(cols) <= 2) {
    for (int r : rows) {
      int mn = -1;//d41
      for (int c : cols) {
        if (mn == -1 || f(r, c, mn)) mn = c;
      }
      row_mins.pb(mn);
    }//d41
  } else if (sz(rows) < sz(cols)) {
    // Reduce
    vector<int> st;
    for (int j : cols) {
      while (!st.empty()) {//d41
        if (f(rows[sz(st) - 1], j, st.back())) st.pop_back
            ();
        else if (sz(st) == sz(rows)) break;
        else { st.push_back(j); break; }
      }
      if (st.empty()) st.pb(j);//d41
    }
    row_mins = smawk(f, rows, st);
  } else {
    // Interpolate
    vector<int> half_rows;//d41
    for (int i = 1; i < sz(rows); i += 2) half_rows.pb(rows
        [i]);
    vector<int> half_row_mins = smawk(f, half_rows, cols);
    vector<int> mn_index;
    for (int i = 0, j = 0; j < sz(cols); j++) {
      while (i < sz(half_row_mins) && cols[j] ==
          half_row_mins[i]) {//d41
        mn_index.pb(j);
        i++;
      }
    }
    for (int i = 0; i < sz(rows); i++) {//d41
      if (i % 2) row_mins.pb(half_row_mins[i / 2]);
      else {
        int l = i == 0 ? 0 : mn_index[i / 2 - 1];
        int r = i == sz(rows) - 1 ? sz(cols) - 1 : mn_index
            [i / 2];
        int mn = cols[l];//d41
        for (int j = l + 1; j <= r; j++) {
          if (f(rows[i], cols[j], mn)) mn = cols[j];
        }
        row_mins.pb(mn);
      }//d41
    }
  }
  return row_mins;
}
//d41
// Min-plus Convolution (b must be convex)
vector<ll> min_plus(vector<ll> a, vector<ll> b) {
  int n = sz(a), m = sz(b);
  auto get = [&](int r, int c) { return a[c] + b[r - c]; };
  auto comp = [&](int r, int c1, int c2) {//d41
    if (r - max(c1, c2) < 0) return c1 < c2;
    if (r - min(c1, c2) >= m) return c2 < c1;
    return get(r, c1) <= get(r, c2);
  };
  vector<int> rows, cols;//d41
  for (int i = 0; i < n + m - 1; i++) rows.pb(i);
  for (int j = 0; j < n; j++) cols.pb(j);
  vector<int> row_mins = smawk(comp, rows, cols);
  vector<ll> res;
  for (int i = 0; i < n + m - 1; i++) res.pb(get(i,
      row_mins[i]));//d41
  return res;
}

// Min-plus Convolution (b must be concave)
vector<ll> max_plus(vector<ll> a, vector<ll> b) {//d41
  for (ll &i : a) i = -i;
  for (ll &i : b) i = -i;
  vector<ll> res = min_plus(a, b);
  for (ll &i : res) i = -i;
  return res;//d41
}
```

XORConvolution.h

Description: Bitwise XOR Convolution
Time: $\mathcal{O}(n \log(n))$

d41d8c, 19 lines

```
void xorfft(vector<ll> &v, bool inv = 0) {//d41
  for (int k = 0; k < bits; k++) {
    for (int i = 0; i < 1 << bits; i++, i += i & 1 << k) {
      ll a = v[i], b = v[i ^ 1 << k];
      v[i] = a + b, v[i ^ 1 << k] = a - b;
    }//d41
  }
  for (int i = 0; i < 1 << bits; i++) v[i] = (v[i] % mod +
      mod) % mod;
  if (inv) {
    ll n = 1; for (int i = 0; i < bits; i++) n = n * 2 %
        mod;
    n = mInv(n);//d41
    for (int i = 0; i < 1 << bits; i++) v[i] = v[i] * n %
        mod;
  }
}
vector<ll> mult(vector<ll> a, vector<ll> b) {
  xorfft(a); xorfft(b); vector<ll> c(1 << bits);//d41
  for (int i = 0; i < 1 << bits; i++) c[i] = a[i] * b[i] %
      mod;
  xorfft(c, 1); return c;
}
```

LinearDiophantineSolver.h

Description: Solves Ax + By = C, outputs a solution to x and y, outputs step size to dx and dy
Time: $\mathcal{O}(fast)$

d41d8c, 11 lines

```
bool diophantine(ll A, ll B, ll C, ll &x, ll &y, ll &dx, ll
    &dy) {//d41
  ll g = exGCD(abs(A), abs(B), x, y);
  if (C % g != 0) return false;
  x *= C / g; y *= C / g;
  if (A < 0) x = -x;
  if (B < 0) y = -y;//d41
  dx = B / g; dy = A / g;
  if (B < 0) dx = -dx;
  if (A < 0) dy = -dy;
  return true;
}//d41
```

LCMConvolution.h

Description: LCM Convolution
Time: about $\mathcal{O}(n \log(n))$

d41d8c, 13 lines

```
// 1-indexed//d41
vector<ll> lcmMult(vector<ll> a, vector<ll> b) {
  vector<ll> l(sz(a), 0), r(sz(a), 0), c(sz(a), 0);
  for (int i = 1; i <= sz(a); i++)
    for (int j = i; j <= sz(a); j += i)
      l[j-1] += a[i-1], r[j-1] += b[i-1];//d41
  for (int g = 1; g <= sz(a); g++) {
    c[g-1] += (l[g-1] % mod) * (r[g-1] % mod) % mod;
    c[g-1] = (c[g-1] % mod + mod) % mod;
    for (int i = g*2; i <= sz(a); i += g) c[i-1] -= c[g-1];
  }//d41
  return c;
}
```

GCDConvolution.h

Description: GCD Convolution
Time: $\mathcal{O}(n \log(n))$

d41d8c, 12 lines

```
// 1-indexed//d41
vector<ll> gcdMult(vector<ll> a, vector<ll> b) {
  vector<ll> c(sz(a), 0);
  for (int g = sz(a); g >= 1; g--) {
    ll l = 0, r = 0;
    for (int i = g; i <= sz(a); i += g)//d41
      l += a[i-1], r += b[i-1], c[g-1] -= c[i-1];
```

```
    c[g-1] += (l % mod) * (r % mod) % mod;
    c[g-1] = (c[g-1] % mod + mod) % mod;
  }
  return c;//d41
}
```

OrConvolution.h

Description: Bitwise OR Convolution and Bitwise AND Convolution
Time: $\mathcal{O}(n \log(n))$

d41d8c, 22 lines

```
void orfft(vector<ll> &v) {//d41
  for (int j = bits - 1; j >= 0; j--)
    for (int i = 0; i < 1 << bits; i++, i += i & 1 << j)
      v[i ^ 1 << j] += v[i];
  for (int i = 0; i < 1 << bits; i++) v[i] %= mod;
}//d41
void orifft(vector<ll> &v) {
  for (int j = 0; j < bits; j++)
    for (int i = 0; i < 1 << bits; i++, i += i & 1 << j)
      v[i ^ 1 << j] -= v[i];
  for (int i = 0; i < 1 << bits; i++) v[i] = (v[i] % mod +
       mod) % mod;//d41
}
vector<ll> orMult(vector<ll> a, vector<ll> b) {
  orfft(a); orfft(b); vector<ll> c(1 << bits);
  for (int i = 0; i < 1 << bits; i++) c[i] = a[i] * b[i] %
       mod;
  orifft(c); return c;//d41
}
vector<ll> andMult(vector<ll> a, vector<ll> b) {
  reverse(all(a)); reverse(all(b));
  vector<ll> c = orMult(a, b);
  reverse(all(c)); return c;//d41
}
```